

■ 课程网站

- <https://kyon-hs.github.io/JLU-COA/>
 - slides等

指令集架构的设计权衡

今天的内容

- 指令集架构的设计权衡

上节课回顾

- 转换层次

- 算法、ISA（指令集体系结构）、微架构

- 基本概念

- 冯·诺依曼模型

- 数据流模型

- ISA 与微架构的比较

回顾：指令集架构（ISA）



■ 指令（Instructions）

- 操作码、寻址方式、数据类型
- 指令类型与格式（Instruction Types and Formats）
- 寄存器、条件码（Registers, Condition Codes）

■ 内存（Memory）

- 地址空间、可寻址性、对齐
- 虚拟内存管理（Virtual memory management）

Intel® 64 and IA-32 Architectures
Software Developer's Manual

Volume 1:
Basic Architecture

■ 其他内容

- 调用、异常/中断处理（Call, Interrupt/Exception Handling）
- 访问控制、优先级/特权（Access Control, Priority/Privilege）
- 输入输出（I/O）：内存映射 I/O vs. 指令驱动 I/O
- 任务/线程管理（Task/thread Management）
- 功耗与热管理（Power and Thermal Management）
- 多线程与多处理器支持（Multi-threading and Multiprocessor support）

回顾：微架构（Microarchitecture）

- 在特定**设计约束与目标（design constraints and goals）**下对 ISA 的具体实现。
- 任何在硬件层面完成且不暴露给软件的设计内容都属于微架构范畴。
 - 流水线（Pipelining）
 - 顺序执行 vs. 乱序执行（In-order vs. Out-of-order Execution）
 - 内存访问调度策略（Memory Access Scheduling Policy）
 - 推测执行（Speculative Execution）
 - 超标量处理（Superscalar Processing, 多指令并发）
 - 时钟门控（Clock Gating）
 - 缓存设计（Caching: 层级、容量、关联度、替换策略）
 - 预取机制（Prefetching）
 - 电压/频率动态调节（Voltage/Frequency Scaling）
 - 错误校正（Error Correction）

ISA 与微架构 (Uarch) 的特性区别

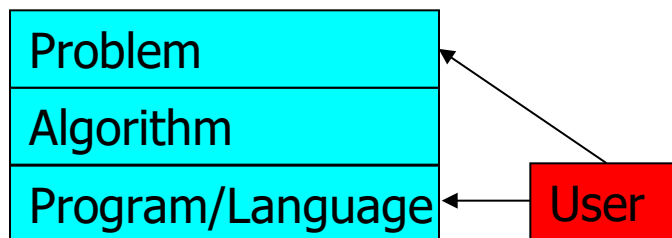
- 加法 (ADD) 指令的操作码
- 通用寄存器 (general purpose registers) 的数量
- 寄存器文件 (register file) 的端口数
- 执行乘法 (MUL) 指令所需的时钟周期数
- 机器是否采用流水线 (pipelined) 指令执行
- 请记住：
 - 微架构 (Microarchitecture)：是在特定设计约束和目标下对 ISA (指令集体系结构) 的实现。

权衡：计算机体系结构的灵魂

- ISA 层面的权衡
- 微架构层面的权衡
- 系统与任务层面的权衡
 - 如何在硬件与软件之间划分工作
- 计算机体系结构是一门科学与艺术，旨在通过恰当的权衡来实现设计目标。
 - 为什么说是“艺术”？

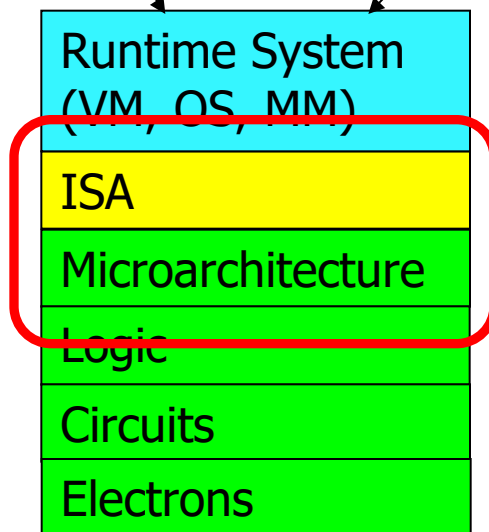
为什么被称为“艺术”？

来自上层的新需求
(向上看)



来自用户的新需求
与个性 (向上看)

来自底层的新问题
与新能力 (向下看)



- 我们并不（完全）了解未来（应用、用户、市场）。
- 而且，未来并不是固定不变的（它会变化）！

来自宏观建筑的类比

- 在宏观建筑中，未来同样不是一成不变的。
- 例如：锅炉房能否在之后被改造成教室？

我们如何适应未来

■ 这是一名优秀计算机体系结构师的重要职责之一。

■ 许多可行的方法

- 敏锐的洞察力与良好的设计
- 善用基本原理与设计原则
 - 高效设计
 - 异构性（多样化体系结构）
 - 可重构性（可根据需求调整架构）
 -
- 充分利用底层技术
-

指令集架构的设计权衡

数十年来的多种指令集架构 (ISAs)

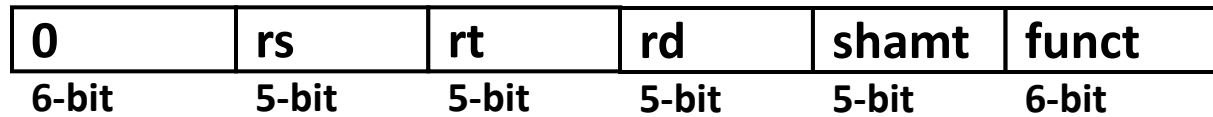
- x86
- PDP-x (Programmed Data Processor, 如 PDP-11)
- VAX
- IBM 360
- CDC 6600
- SIMD 类型 ISA: 如 CRAY-1、Connection Machine
- VLIW 类型 ISA: 如 Multiflow、Cydrome、IA-64 (EPIC)
- PowerPC / POWER
- RISC 类型 ISA: 如 Alpha、MIPS、SPARC、ARM
- 思考: 这些 ISA 的根本区别是什么?
 - 指令是如何定义和执行的? (how instructions are specified and what they do)
 - 指令的复杂度有多高? (how complex are the instructions)

指令 (Instruction)

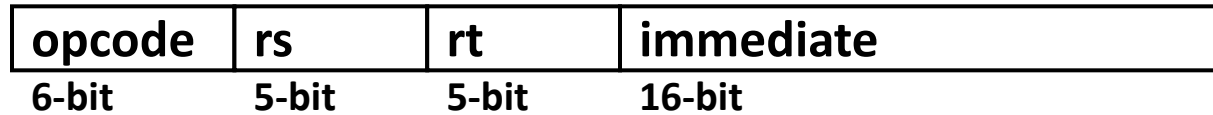
- 是**硬件与软件接口 (HW/SW interface) **的基本元素。
- 组成部分 (Consists of)
 - Opcode (操作码) : 描述指令要执行的操作 (what the instruction does) 。
 - Operands (操作数) : 指明操作对象 (who it is to do it to) 。
- 示例: Alpha ISA 中的指令格式

31	26	25	21	20	16	15	5	4	0	
Opcode		Number								PALcode Format
Opcode		RA		Disp						Branch Format
Opcode		RA		RB		Disp				Memory Format
Opcode		RA		RB		Function			RC	Operate Format

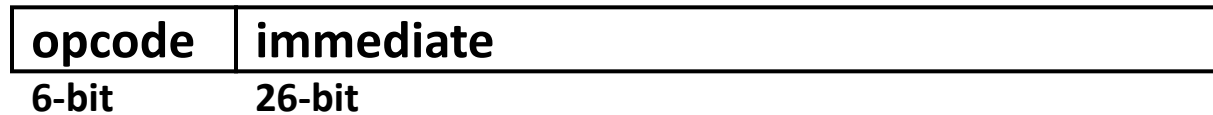
MIPS



R-type



I-type



J-type

ARM

3 3 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0																																						
Cond	0	0	I	Opcode				S	Rn				Rd				Operand 2														Data Processing / PSR Transfer							
Cond	0	0	0	0	0	0	0	A	S	Rd				Rn				Rs	1	0	0	1	Rm				Multiply											
Cond	0	0	0	0	1	U	A	S	RdHi				RdLo				Rn				1	0	0	1	Rm				Multiply Long									
Cond	0	0	0	1	0	B	0	0	Rn				Rd				0	0	0	0	1	0	0	1	Rm				Single Data Swap									
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn				Branch and Exchange										
Cond	0	0	0	P	U	0	W	L	Rn				Rd				0	0	0	0	1	S	H	1	Rm				Halfword Data Transfer: register offset									
Cond	0	0	0	P	U	1	W	L	Rn				Rd				Offset				1	S	H	1	Offset				Halfword Data Transfer: immediate offset									
Cond	0	1	I	P	U	B	W	L	Rn				Rd				Offset														Single Data Transfer							
Cond	0	1	1																												1					Undefined		
Cond	1	0	0	P	U	S	W	L	Rn				Register List																			Block Data Transfer						
Cond	1	0	1	L	Offset																																	Branch
Cond	1	1	0	P	U	N	W	L	Rn				CRd				CP#				Offset										Coprocessor Data Transfer							
Cond	1	1	1	0	CP Opc				CRn				CRd				CP#				CP	0	CRm				Coprocessor Data Operation											
Cond	1	1	1	0	CP Opc				CRn				Rd				CP#				CP	1	CRm				Coprocessor Register Transfer											
Cond	1	1	1	1	Ignored by processor																																	Software Interrupt
3 3 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0																																						

Figure 4-1: ARM instruction set formats

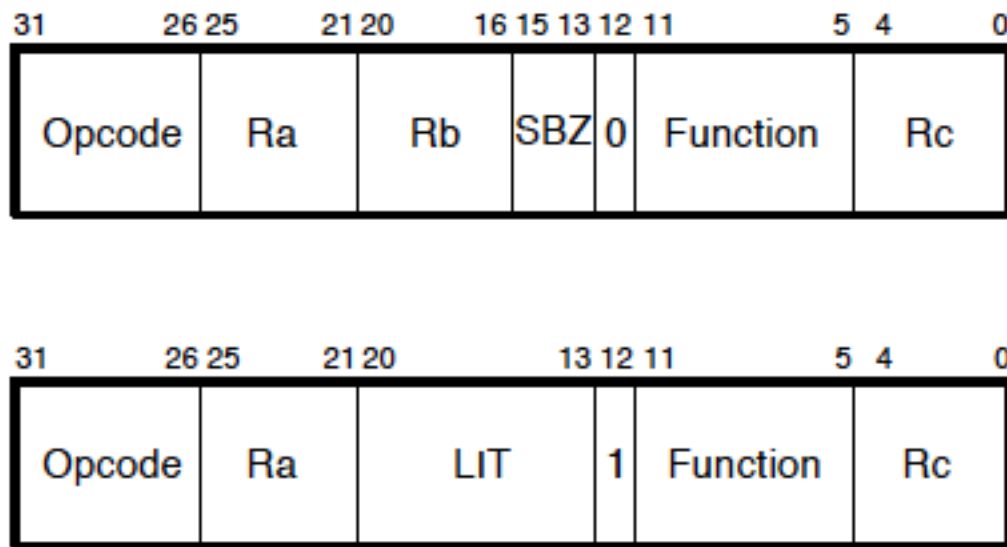
指令集、编码与规范

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			A	op.spec				
AND ⁺	0101				DR			SR1			A	op.spec				
BR	0000				n	z	p	PCOffset9								
JMP	1100				000			BaseR			000000					
JSR(R)	0100				A	operand.specifier										
LDB ⁺	0010				DR			BaseR			boffset6					
LDW ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCOffset9								
RTI	1000				000000000000											
SHF ⁺	1101				DR			SR			A	D	amount4			
STB	0011				SR			BaseR			boffset6					
STW	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
XOR ⁺	1001				DR			SR1			A	op.spec				
not used	1010															
not used	1011															

- 示例：LC-3b 指令集（ISA）
- x86 手册（x86 Manual）
- 为什么会存在“未使用的指令”（unused instructions）？ → 预留空间以便未来扩展或保持编码对齐。

Alpha 架构中的位引导

Figure 3-4: Operate Instruction Format



规则 (Bit Steering Rule)

- 当指令的第 12 位 (bit <12>) = 0 时: → Rb 字段表示一个源寄存器 (Source Register Operand)。
- 当指令的第 12 位 (bit <12>) = 1 时: → Rb 字段被重新解释为一个 8 位的立即数 (literal constant), 由位 <20:13> 形成。→ 该立即数会**零扩展 (zero-extend)**到 64 位。→ 它表示一个 0 - 255 范围内的正整数。

指令集架构（ISA）的组成元素

■ 指令排序模型（Instruction sequencing model）

- 控制流（Control flow） vs. 数据流（Data flow）
- 权衡取舍？（Tradeoffs?）

■ 指令处理风格（Instruction processing style）

- 规定一条指令操作的“操作数”（operands）数量以及其操作方式
- 0、1、2、3 地址机器：
 - 0 地址机器：堆栈机（stack machine）
 - 1 地址机器：累加器机（accumulator machine）
 - 2 地址机器：双操作数机（2-operand machine）
 - 3 地址机器：三操作数机（3-operand machine）

■ 权衡取舍（Tradeoffs）

- 较大的操作指令 vs. 更多的执行操作
- 代码大小 vs. 执行时间 vs. 片上内存空间（on-chip memory space）

一个例子：堆栈机 (Stack Machine)

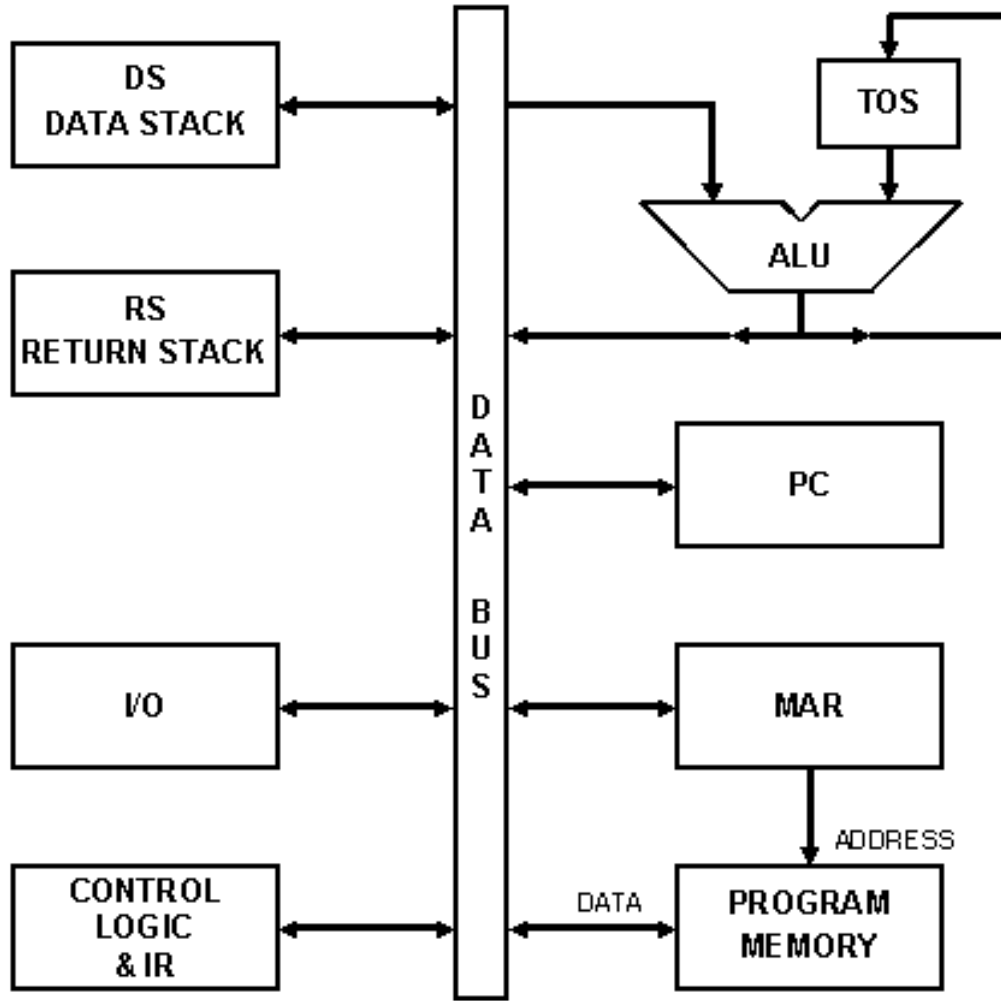
■ 优点 (+)

- 指令体积小（操作指令不需要操作数）
 - 逻辑更简单 (Simpler logic)
 - 代码更紧凑 (Compact code)
- 过程调用效率高：所有参数都在堆栈中
 - 不需要额外的周期来传递参数 (No additional cycles for parameter passing)

■ 缺点 (—)

- 不容易用“后缀表示法”表达的计算，难以映射到堆栈机上
 - 无法同时对多个值进行操作（一次只能处理堆栈顶部的 N 个值）
 - 灵活性较差 (Not flexible)

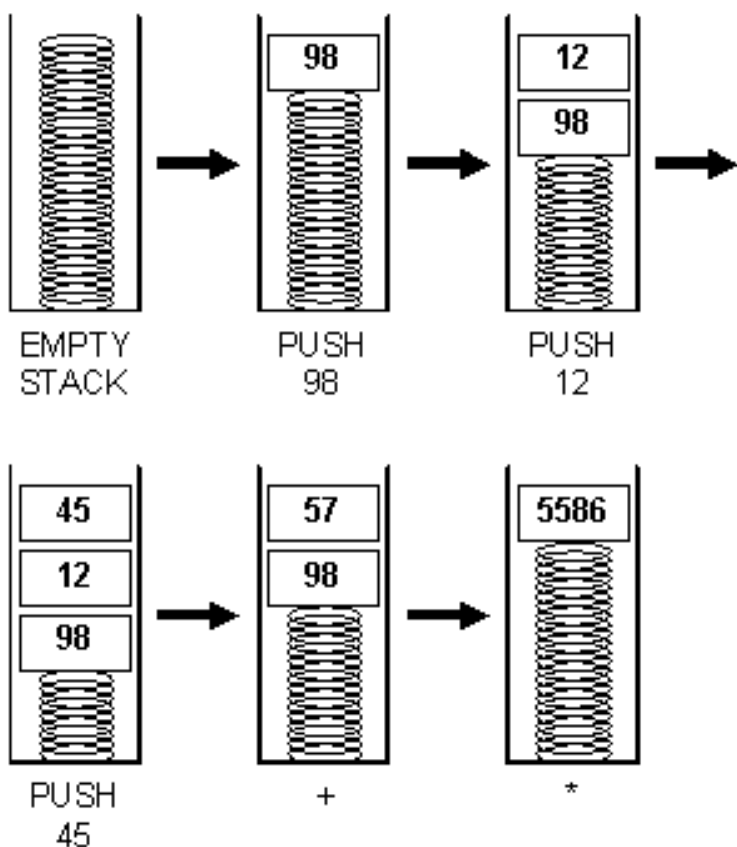
一个例子：堆栈机 (Stack Machine)



Koopman, “**Stack Computers: The New Wave**,” 1989.

Figure 3.1 -- The canonical stack machine.

一个例子：堆栈机 (Stack Machine)



■ Koopman, “Stack Computers: The New Wave,” 1989.

Figure 3.2 -- An example stack machine.

指令集架构 (ISA) 的组成元素

■ 指令 (Instructions)

- 操作码 (Opcode)
- 操作数说明符 (Operand specifiers, 寻址方式 addressing modes)
 - 如何获取操作数?
 - 为什么会有不同的寻址方式?

■ 数据类型 (Data types)

- 定义: 信息的表示形式, 用于描述指令所操作的数据表示方式
- 整数 (integer)、浮点数 (floating point)、字符 (character)、二进制 (binary)、十进制 (decimal)
- 双向链表 (doubly linked list)、队列 (queue)、字符串 (string)、位向量 (bit vector)、堆栈 (stack)
 - **VAX:** INSQUEUE 和 REMQUEUE 指令用于操作双向链表或队列; FINDFIRST
 - **Digital Equipment Corp:** *VAX11 780 Architecture Handbook*, 1977
 - **X86:** SCAN 操作码可对字符串进行操作; PUSH / POP

数据类型的权衡

- 在 ISA 中拥有更多或更高级别的数据类型有什么好处？
- 又有什么缺点？
- 思考角度： 编译器 / 程序员 vs. 微体系结构设计者
- 语义鸿沟（Semantic Gap）的概念
 - 数据类型与语义层次（或指令复杂度）紧密耦合
- 示例：早期 RISC 架构 vs. Intel 432
 - 早期 RISC： 仅支持整数数据类型
 - Intel 432： 支持对象数据类型，是基于“能力”（capability-based）的机器

指令集架构（ISA）的组成元素

■ 内存组织（Memory organization）

- 地址空间（Address space）：内存中有多少个可唯一标识的位置。
- 可寻址性（Addressability）：每个可唯一标识的位置可以存储多少数据。
 - 按字节寻址（Byte addressable）：大多数 ISA 是按字节寻址，字符通常是 8 位（8 bits）。
 - 按位寻址（Bit addressable）：例如 Burroughs 1700。为什么？
 - 按 64 位寻址（64-bit addressable）：某些超级计算机采用此方式。为什么？
 - 按 32 位寻址（32-bit addressable）：最早的 Alpha 处理器使用此方式。
- 对虚拟内存的支持（Support for virtual memory）

指令集架构（ISA）的组成元素

■ 寄存器 (Registers)

- 数量 (How many)
- 每个寄存器的大小 (Size of each register)

■ 为什么寄存器是一个好主意？ (Why is having registers a good idea?)

- 因为程序表现出一种称为局部性 (data locality) 的特性。
- 最近生成或访问过的值很可能会被再次使用 (时间局部性, temporal locality)。
 - 将该值存储在寄存器中, 可以避免每次需要使用时都去访问内存。

程序员可见的状态

M[0]
M[1]
M[2]
M[3]
M[4]
M[N-1]

内存 (Memory)

一组由地址索引的存储位置组成的数组。



寄存器 (Registers)

在 ISA (指令集体系结构) 中被赋予特定名称 (而非地址)。

分为 通用寄存器 (general purpose) 和 专用寄存器 (special purpose)

程序计数器 (Program Counter, PC)
存储当前正在执行指令的内存地址。

寄存器体系结构的演化

■ 累加器 (Accumulator)

- 来源于早期“加法机 (adding machine)”时代的遗产。

■ 累加器 + 地址寄存器 (Accumulator + Address Registers)

- 需要通过寄存器间接寻址 (register indirection)。
- 起初，地址寄存器是专用寄存器 (special-purpose)，即只能加载一个地址，用于间接寻址。
- 随后，地址运算 (arithmetic on addresses) 也得到了支持。

■ 通用寄存器 (General Purpose Registers, GPR)

- 所有寄存器都可用于各种用途。
- 寄存器数量从早期的少量增加到：
 - 常见的 RISC 架构拥有 32 个寄存器；
 - Intel IA-64 体系结构中多达 128 个寄存器。

指令类别 (Instruction Classes)

■ 运算指令 (Operate instructions)

- 处理数据：执行算术和逻辑运算。
- 操作流程：取操作数 (fetch operands)、计算结果 (compute result)、存储结果 (store result)。
- 隐式顺序控制流 (Implicit sequential control flow)：指令按顺序依次执行。

■ 数据传输指令 (Data movement instructions)

- 在内存、寄存器与 I/O 设备之间移动数据。
- 隐式顺序控制流：顺序执行数据移动操作。

■ 控制流指令 (Control flow instructions)

- 改变指令的执行顺序。
(例如：跳转、分支、调用、返回等指令)

指令集架构（ISA）的组成元素

- 加载/存储（Load/Store） vs. 内存/内存（Memory/Memory）体系结构
 - 加载/存储体系结构（Load/Store Architecture）
 - 运算指令只作用于寄存器。
 - 示例：MIPS、ARM 以及许多 RISC（精简指令集）架构。
 - 内存/内存体系结构（Memory/Memory Architecture）
 - 运算指令可以直接作用于内存位置。
 - 示例：x86、VAX 以及许多 CISC（复杂指令集）架构。

指令集架构（ISA）的组成元素

■ 寻址模式（Addressing modes）用于指定如何获得操作数。

- 立即数寻址（Absolute）`LW rt, 10000`
使用立即数（immediate value）作为地址。
- 寄存器间接寻址（Register Indirect）`LW rt, (r_base)`
使用寄存器 `GPR[r_base]` 的内容作为地址。
- 带位移或基址寻址（Displaced or Based）`LW rt, offset(r_base)`
使用 `offset + GPR[r_base]` 作为地址。
- 索引寻址（Indexed）`LW rt, (r_base, r_index)`
使用 `GPR[r_base] + GPR[r_index]` 作为地址。
- 内存间接寻址（Memory Indirect）`LW rt, ((r_base))`
使用 `M[ GPR[r_base] ]` 的值作为地址。
- 自动递增/递减寻址（Auto Increment/Decrement）`LW rt, (r_base)`
使用 `GPR[r_base]` 作为地址，但每次取数后会自动递增或递减 `GPR[r_base]`。

不同寻址方式的优点是什么？

- 又一个程序员与微体系结构设计师（microarchitect）之间的权衡例子
- 拥有更多寻址方式的优点：
 - 可以更好地将高级语言结构映射到机器层面
一些数据访问用不同的寻址方式表达更自然，从而减少指令数量和代码体积。
例如：
 - 数组访问（使用自动递增模式 *autoincrement mode*）
 - 间接寻址（指针追踪 *pointer chasing*）
 - 稀疏矩阵访问（*sparse matrix accesses*）
- 缺点：
 - 编译器需要做更多工作
 - 微体系结构设计师也需要做更多工作

ISA 正交性 (ISA Orthogonality)

■ 正交的 ISA (Orthogonal ISA)

- 所有寻址方式 (addressing modes) 都可以与所有指令类型 (instruction types) 组合使用。
- 示例: VAX 架构
 - 约 13 种寻址方式 × 超过 300 条操作码 × 整数与浮点格式 (integer and FP formats)

■ 这对谁有好处?

➤ 程序员 (programmers) ——因为他们拥有更大的灵活性, 能用多种方式表达相同操作。

■ 这对谁不利?

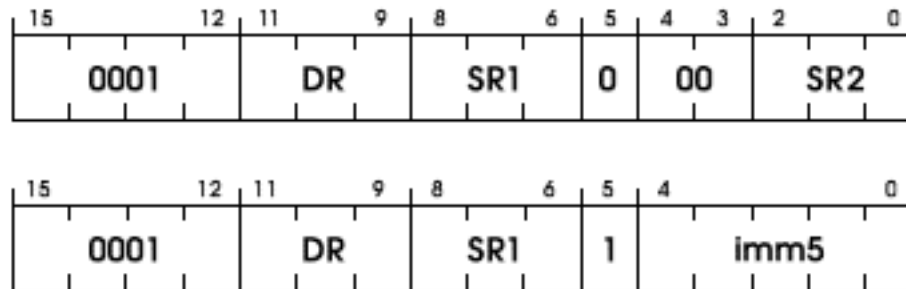
➤ 微体系结构设计师 (microarchitects) ——因为硬件实现更复杂, 解码和执行逻辑更难优化。

LC-3b 的 ISA 是正交的吗？

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			A	op.spec				
AND ⁺	0101				DR			SR1			A	op.spec				
BR	0000				n	z	p	PCoffset9								
JMP	1100				000			BaseR			000000					
JSR(R)	0100				A	operand.specifier										
LDB ⁺	0010				DR			BaseR			boffset6					
LDW ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCoffset9								
RTI	1000				000000000000											
SHF ⁺	1101				DR			SR			A	D	amount4			
STB	0011				SR			BaseR			boffset6					
STW	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
XOR ⁺	1001				DR			SR1			A	op.spec				
not used	1010															
not used	1011															

LC-3b: ADD 指令的寻址方式

Encodings

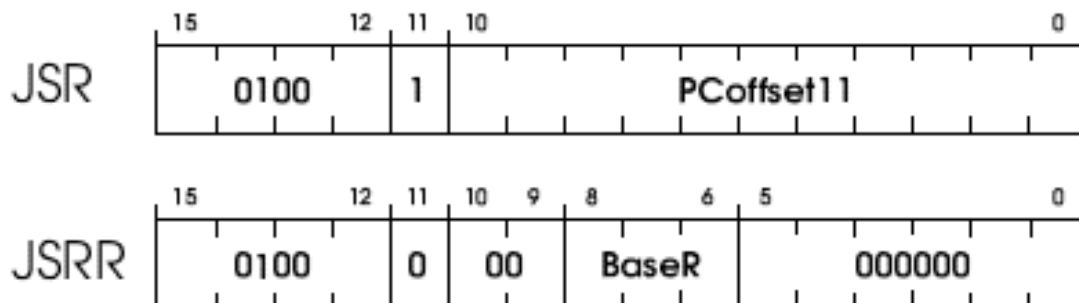


Operation

```
if (bit[5] == 0)
    DR = SR1 + SR2;
else
    DR = SR1 + SEXT(imm5);
setcc();
```

LC-3b: JSR (R) 指令的寻址方式

Encodings



Operation

```
R7 = PC†;  
if (bit[11] == 0)  
    PC = BaseR;  
else  
    PC = PC† + LSHF(SEXT(PCOffset11), 1);
```

■ **描述 (Description)**：首先，递增后的 PC（程序计数器）被保存到 R7，用于从子程序返回。然后，处理器无条件跳转到子程序的起始地址，该地址由两种方式之一确定：当 $\text{bit}[11] == 0$ 时：地址从寄存器 BaseR 中取得（即 JSRR 模式）。当 $\text{bit}[11] == 1$ 时：地址通过 $\text{PC} + (\text{符号扩展的 PCOffset11 左移 } 1 \text{ 位})$ 计算得到（即 JSR 模式）。